

TP2 - Ledger

Objetivo del trabajo:

Desarrollar un sistema de libro contables (ledger) que registre transacciones de diferentes monedas entre usuarios.

Componentes principales

Usuarios

Se debe generar una entidad usuario.

Los mismos tendran id (numerico), nombre de usuario (string), fecha de nacimiento (date), fecha de creación de usuario (date), fecha de edición del usuario.

Validaciones:

- id unico, primary key.
- El nombre de usuario debe ser único.
- El usuaio debe tener más de 18 años al crear la cuenta.
- El nombre de usuario debe poder editarse, al editarse, debe ser distinto al que tenía.
- Al momento de borrar un usuario, solo debe realizarse si el usuario no tiene ninguna transacción asociada.
- Todos los campos son obligatorios.

Comandos:

```
./ledger crear_usuario -n=<nombre-de-usuario> -b=<fecha-nacimiento>
./ledger editar_usuario -id=<id-usuario> -n=<nuevo-nombre-de-usuario>
./ledger borrar_usuario -id=<id-usuario>
./ledger ver_usuario -id=<id-usuario>
```

Monedas

Se debe generar una entidad moneda.

Las mismas tendran id (numerico), nombre de moneda (string), precio en dolares (float), fecha de creación de moneda (date), fecha de edición de moneda (date).

Validaciones:

- id unico, primary key.
- El precio no puede ser negativo.
- El nombre de la moneda es único, no puede editarse, debe estar en mayúsculas y ser de mínimo 3 letras y máximo 4.
- Todos los campos son obligatorios.
- Al momento de borrar una moneda, solo debe realizarse si la moneda no tiene ninguna transacción asociada.

Comandos:

```
./ledger crear_moneda -n=<nombre-de-moneda> -p=<precio-respecto-dolar>
./ledger editar_moneda -id=<id-moneda> -p=<nuevo-precio-respecto-dolar>
./ledger borrar_moneda -id=<id-moneda>
./ledger ver_moneda -id=<id-moneda>
```

Transacciones

Se debe generar una entidad transaccion.

Las mismas tendrán id (numerico), timestamp, moneda origen (foreign_key a monedas), moneda destino (foreign key a monedas), monto (float), cuenta origen (foreign key a usuarios), cuenta destino (foreign key a usuarios), tipo (string).

Validaciones:

- id unico, primary key.
- Solo puede deshacerse una transaccion si es la última de lo/los usuarios asociados. Para deshacer una transacción, se debe generar una nueva, pero opuesta.
- Solo se pueden realizar transaccion desde y hacia cuentas que han sido creadas previamente.
-

Comandos:

```
./ledger alta_cuenta -u=<id-usuario> -m=<id-moneda>
./ledger realizar_transferencia -o=<id-usuario-origen> -d=<id-usuario-destino>
./ledger realizar_swap -u=<id-usuario> -mo=<id-moneda-origen> -md=<id-moneda-destino>
./ledger deshacer_transaccion -id=<id-transaccion>
./ledger ver_transaccion -id=<id-transaccion>
```

Consideraciones

- Los comandos del TP1 dejen seguir funcionando, solo que tomando la información desde la nueva estructura.
- En los comando `ver_***` , queda a consideración del alumno la información a mostrar.
- En caso de no poder realizar el comando, debe mostrarse un error por consola, con el siguiente formato:

```
{:error, <comando>: <descripción del error>}
```

Por ejemplo si se quiere dar de alta un usuario con un nombre que ya está en uso, debe mostrarse:

```
{:error, crear_usuario: "El nombre de usuario ya se encuentra en uso."}
```

Las descripciones de error pueden ser cualquier texto que explique lo sucedido.

- El trabajo se entregará por Algotrón en un .zip, excluyendo todo lo que se ignora en un .gitignore de elixir típico.
- Se debe documentar como compilar y correr el proyecto en el README.
- El proyecto debe tener un test coverage de al menos el 90%.
- Para generar un ejecutable, se recomienda seguir la siguiente guía: <https://elixirschool.com/en/lessons/intermediate/escripts> (<https://elixirschool.com/en/lessons/intermediate/escripts>)
- Fecha límite de entrega 15/10/2025 a las 23.59.